

Lecture Unit 5.2

Floating point exceptions

Floating point exceptions

- There are some situations in which floating point arithmetic can have problems:
 - What does $1.0/0.0$ equal?
 - What does $0.0/0.0$ equal?
 - What if we assign the square root of a negative number to a float or double?
 - What happens if we assign a result which is too large or too small for the floating point format to represent?

IEEE 754 double precision

- If the exponent bits are neither all zeros or all ones then the number is said to be a “normalised” floating point number
- A normalised floating point number x is represented by:

$$x = \pm (1.b_1 b_2 \dots b_{52})_2 \times 2^{(a_1 a_2 \dots a_{11})_2 - 1023}$$

where $b_1 b_2 \dots b_{52}$ are the mantissa bits, $a_1 a_2 \dots a_{11}$ are the exponent bits, and $(\dots)_2$ means a binary number

- If the exponent bits are all zeros or all ones then this represents an “exceptional” value

Subnormal numbers

- If the exponent bits are all zeros then the number is said to be a “subnormal” floating point number
- A subnormal floating point number x is represented by:

$$x = \pm (0.b_1 b_2 \dots b_{52})_2 \times 2^{(a_1 a_2 \dots a_{11})_2 - 1022}$$

- where $b_1 b_2 \dots b_{52}$ are the mantissa bits, $a_1 a_2 \dots a_{11}$ are the exponent bits, and $(\dots)_2$ means a binary number
- There are leading zeros in the mantissa so precision has been lost
- Subnormal numbers expand the range around zero and allow for more gradual underflow

Infinites and NaNs

- If all the exponent bits are ones and all the mantissa bits are zero then the number is
 - `inf` representing $+\infty$ or
 - `-inf` representing $-\infty$
- If all the exponent bits are ones and not all the mantissa bits are zero then this represents
 - `NaN` which means “Not a Number”

Workshop3

- Change the velocity (`velx`) to `-0.01`
- Now compile and run the program. This will not work correctly!

Question 6: What are the values of `u` in the file `final.dat`?

```
0.017450 inf
0.017550 -inf
0.017650 inf
0.017750 -inf
0.017850 inf
0.017950 -inf
0.018050 inf
0.018150 -inf
0.018250 inf
0.018350 -inf
0.018450 inf
0.018550 -inf
0.018650 inf
0.018750 -inf
0.018850 inf
0.018950 -nan
0.019050 -nan
0.019150 -nan
0.019250 -nan
0.019350 -nan
0.019450 -nan
0.019550 -nan
0.019650 -nan
0.019750 -nan
0.019850 -nan
0.019950 -nan
0.020050 -nan
0.020150 -nan
```

Infinites and NaNs (not a number)
The calculation has gone badly wrong!

Examples

```
#include <stdio.h>
#include <math.h>

int main (){

    float zero    = 0.0;
    float one     = 1.0;
    float minusOne = -1.0;
    float large   = 1.0e25;
    float tiny    = 1.0e-25;
    float num_single;
    double num_double;

    /* Overflow */
    num_single = large * large;
    printf("single: large^2 = %g\n", num_single);

    /* OK */
    num_double = (double) large * (double) large;
    printf("double: large^2 = %g\n\n", num_double);

    /* Underflow */
    num_single = tiny*tiny;
    printf("single: tiny^2 = %g\n", num_single);

    /* OK */
    num_double = (double) tiny * (double) tiny;
    printf("double: tiny^2 = %g\n\n", num_double);

    /* Divide by zero*/
    printf("1.0/0.0 = %f\n\n", one/zero);

    /* Invalid */
    printf("0.0/0.0 = %f\n\n", zero/zero);

    /* Invalid */
    printf("sqrt(-1.0) = %f\n\n", sqrt(minusOne));

    return 0;
}
```

> gcc -o fpe fpe.c -lm
> ./fpe

single: large^2 = inf
double: large^2 = 1e+50

single: tiny^2 = 0
double: tiny^2 = 1e-50

1.0/0.0 = inf

0.0/0.0 = -nan

sqrt(-1.0) = -nan

Floating point exception types

- There are five floating point exceptions specified by IEEE 754
 - Overflow: `inf` or `-inf`
 - Underflow: `0` or subnormal number
 - Divide by zero: `inf` or `-inf`
 - Invalid: `nan`
 - Inexact: mathematically inexact, returns rounded result
- Floating point exceptions set a flags in the hardware and can be trapped for debugging

Unit Summary

- In this unit we have looked at what happens when floating point goes wrong!
- There are different types of exception which may return exceptional values

Exception types:

Invalid

Divide by zero

Overflow

Underflow

Inexact

Exceptional values:

Not a number (**NaN**)

Infinity (**inf**)

Sub-normal numbers